

EXPERIMENT – 3

Fundamentals of Errors and Debugging

1. Introduction

Errors are common while developing a computer program. An error may prevent a program from compiling, cause it to stop during execution, or produce an incorrect result. Debugging is the process of locating such problems and correcting them.

Important Terms

Error:

An error is a mistake in a program that may prevent it from working as intended.

Example: Using + when multiplication * was required.

Bug:

A bug is a defect in a program that produces an unexpected or incorrect result.

Debugging:

Debugging is the step-by-step process of identifying, examining and fixing bugs in a program.

2. Types of Errors in C

Error Type

Meaning

Common Reason

Example

Syntax Error

The program does not follow the rules of C syntax.

Missing ;, brackets or incorrect keywords

`int n = 20`

Compilation/Type Error

An operation is performed using an unsuitable data type or declaration.

Wrong type assignment or undeclared variable

`int *p = "Hello";`

Linker Error

Object files cannot be properly combined to create the final executable.

Missing function definition or library

Undefined reference to a function

Runtime Error

Error occurs while the program is running.

Division by zero or invalid memory access

int x = 5/0;
Logical Error
Program executes but gives an incorrect answer.
Incorrect formula or condition
Wrong loop limit

CASE 1: IDENTIFICATION OF A SYNTAX ERROR

Aim

To locate a syntax error in a C program and correct it so that the program can compile successfully.

Program Containing Error

```
#include <stdio.h>
```

```
int main()
{
    int number = 25
    printf("Number = %d\n", number);

    return 0;
}
```

Compilation Result

```
$ gcc case1.c
```

```
case1.c:6:5: error: expected ',' or ';' before 'printf'
```

Error Analysis

The declaration of number does not end with a semicolon. Since every normal C statement must be terminated properly, the compiler reports an error when it reaches the printf() statement.

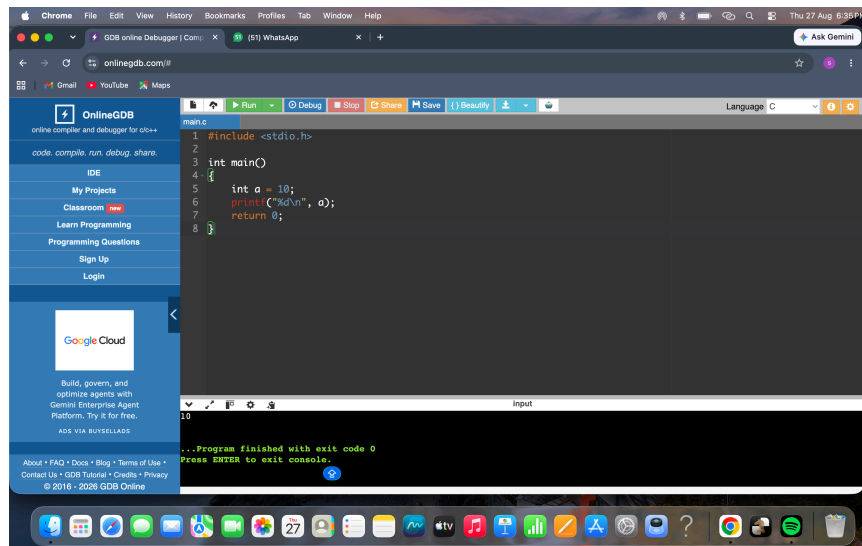
Correct Program

```
#include <stdio.h>
```

```
int main()
{
    int number = 25;
    printf("Number = %d\n", number);

    return 0;
}
```

Output



Number = 25

Conclusion

The error was caused by a missing semicolon after the variable declaration. After inserting ;, the program compiled and executed correctly.

CASE 2: IDENTIFYING COMPILER WARNINGS

Aim

To observe compiler warnings using the -Wall and -Wextra options.

Program

```
#include <stdio.h>
```

```
int main()
{
    int marks;

    printf("Program executed successfully.\\n");

    return 0;
}
```

Normal Compilation

```
$ gcc case2.c -o case2
```

```
$ ./case2
```

Program executed successfully.

The program runs normally because the unused variable does not prevent compilation.

Compilation With Warning Flags

```
$ gcc -Wall -Wextra case2.c -o case2
```

case2.c: In function 'main':

case2.c:6:9: warning: unused variable 'marks'

Explanation

The compiler detects that marks has been declared but its value is never used. This is a warning rather than a fatal error. Warning options are useful because they can reveal possible mistakes that may otherwise remain unnoticed.

Output

```
main.c:5:9: warning: unused variable 'a' [-Wunused-variable]
```

```
    5 |     int a;  
      |         ^
```

```
Hello
```

```
...Program finished with exit code 0
```

```
Press ENTER to exit console.
```

Program executed successfully.

Conclusion

Compiler warnings generally do not stop a valid program from executing. Using -Wall and -Wextra helps programmers identify suspicious or unnecessary code and improves code quality.

CASE 3: DEBUGGING A LOGICAL ERROR USING PRINTF()

Aim

To trace a logical mistake in a loop by displaying intermediate variable values with printf().

Program With Logical Error

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, sum = 0;
```

```
    for (i = 0; i < 5; i++)
```

```
    {
```

```
        printf("DEBUG: Current i = %d\n", i);
```

```
        sum = sum + i;
```

```
        printf("DEBUG: Current sum = %d\n", sum);
```

```

    }

    printf("Final Sum = %d\n", sum);

    return 0;
}

```

Problem

The purpose of the program is to calculate:

$1 + 2 + 3 + 4 + 5$

However, the loop begins from 0 and stops when i becomes 5.

Therefore, it actually adds:

$0 + 1 + 2 + 3 + 4 = 10$

The expected answer is:

$1 + 2 + 3 + 4 + 5 = 15$

Debugging Technique

The printf() statements inside the loop display the value of i and the updated value of sum after every iteration. By observing these values, the incorrect loop range can easily be identified.

Correct Loop

for (i = 1; i <= 5; i++)

Correct Program

```
#include <stdio.h>
```

```

int main()
{
    int i, sum = 0;

    for (i = 1; i <= 5; i++)
    {
        printf("DEBUG: i = %d\n", i);

        sum = sum + i;

        printf("DEBUG: sum = %d\n", sum);
    }

    printf("Final Sum = %d\n", sum);

    return 0;
}

```

Output

DEBUG: i = 1
DEBUG: sum = 1
DEBUG: i = 2
DEBUG: sum = 3
DEBUG: i = 3
DEBUG: sum = 6
DEBUG: i = 4
DEBUG: sum = 10
DEBUG: i = 5
DEBUG: sum = 15
Final Sum = 15

Conclusion

printf() debugging makes it easier to observe the changing values of variables during execution. It helped us locate the incorrect loop boundary and correct the logical error.

3. INTRODUCTION TO GDB

What is GNU and GDB?

GNU:

GNU is a free and open-source software project that provides many tools and programs used in Unix-like operating systems.

GDB (GNU Debugger):

GDB is a debugging tool used to examine and control the execution of programs. It allows programmers to set breakpoints, inspect variables, execute programs step-by-step and locate logical problems.

Important GDB Commands

Command

Short Form

Purpose

break <line/function>

b

Creates a breakpoint at a particular location.

run

r

Starts program execution.

next

n

Executes the next source line without entering a called function.

step

s

Moves to the next line and enters a function call if present.

print <variable>

p

Shows the current value of a variable or expression.

display <variable>

disp

Automatically displays a variable after each step.

continue

c

Continues execution until another breakpoint or program termination.

list

l

Displays source-code lines around the current location.

info locals

info loc

Shows local variables and their values.

backtrace

bt

Displays the current function-call stack.

info breakpoints

info b

Shows the breakpoints currently created.

delete <id>

d

Removes a breakpoint.

disable <id>

dis

Temporarily turns off a breakpoint.

enable <id>

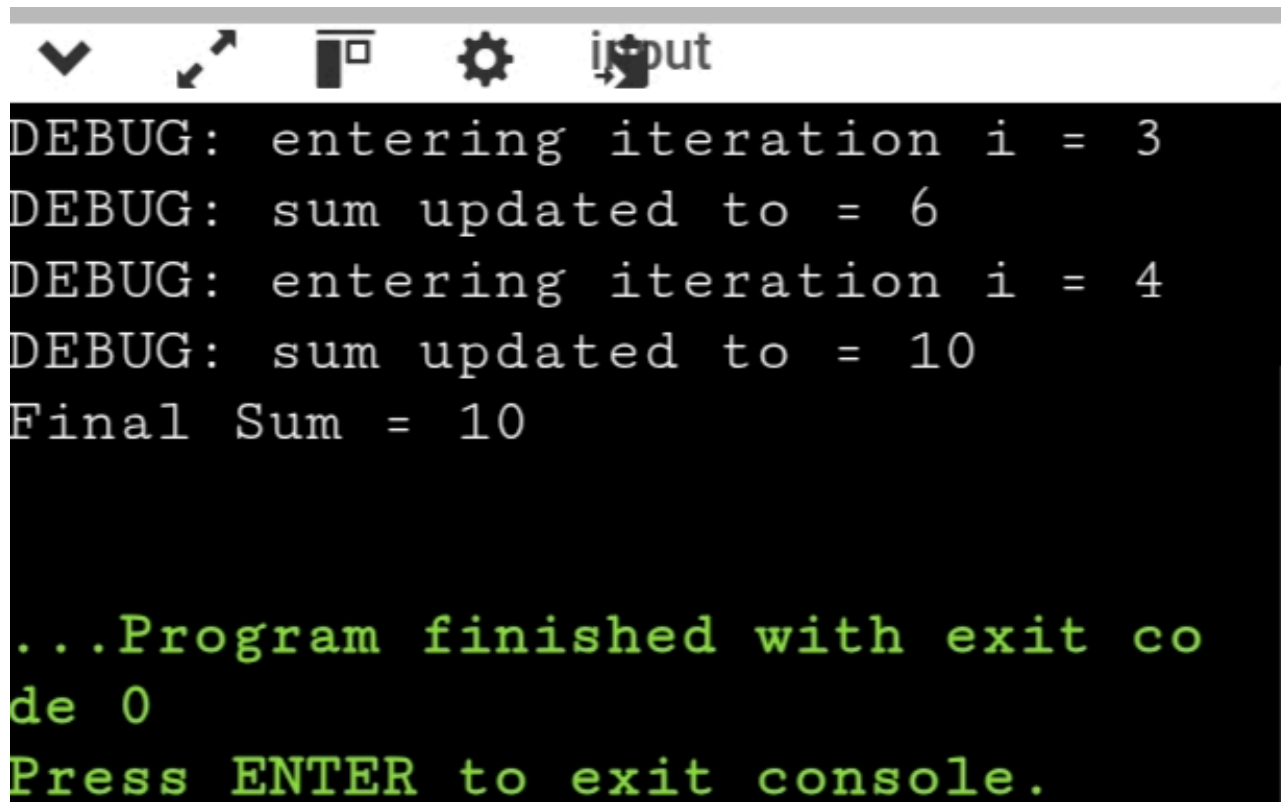
en

Activates a disabled breakpoint.

quit

q

Closes the GDB session.



```
DEBUG: entering iteration i = 3
DEBUG: sum updated to = 6
DEBUG: entering iteration i = 4
DEBUG: sum updated to = 10
Final Sum = 10

...Program finished with exit code 0
Press ENTER to exit console.
```

CASE 4: DEBUGGING A LOGICAL ERROR USING GDB

Aim

To identify and correct a logical error in a C program with the help of GDB.

Program

```
#include <stdio.h>
```

```
int main()
```

```
{
    int i, sum = 0;
```

```
    for (i = 1; i < 5; i++)
```

```
    {
        sum = sum + i;
    }
```

```
    printf("Sum = %d\n", sum);
```

```
    return 0;
```

```
}
```

Problem Identification

The loop condition is:

$i < 5$

Hence the loop executes for:

1, 2, 3, 4

The program calculates:

$1 + 2 + 3 + 4 = 10$

But if the requirement is to add numbers from 1 to 5, the required result is:

$1 + 2 + 3 + 4 + 5 = 15$

GDB Debugging

First compile the program with debugging information:

\$ gcc
Start
\$

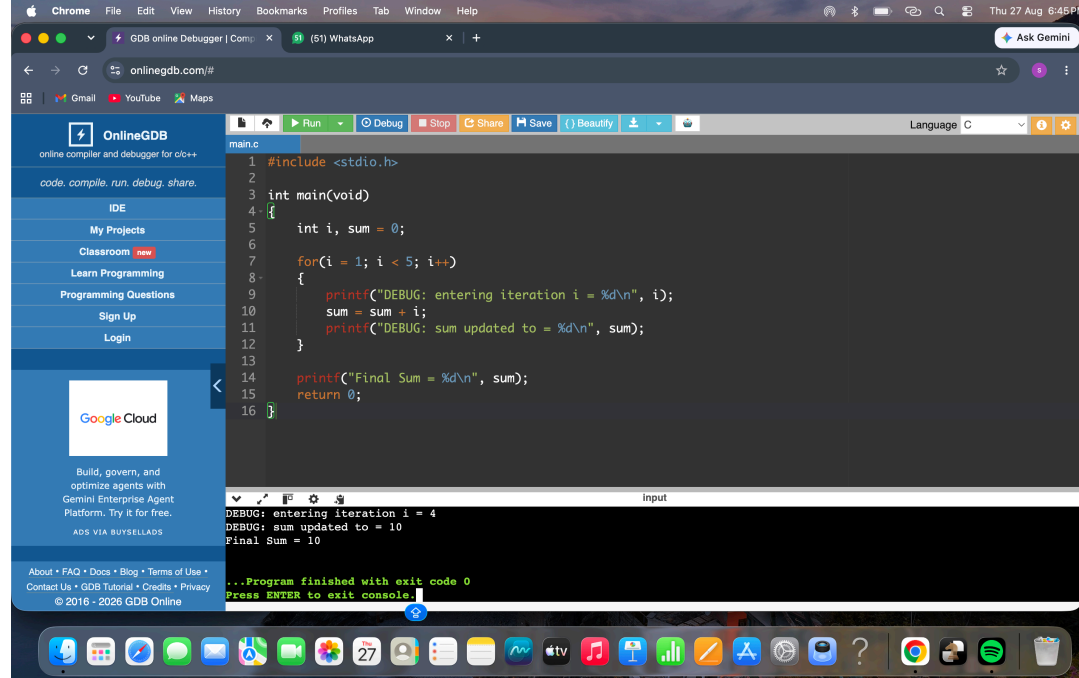
(gdb)
Set a
(gdb)
Run
(gdb)
Check
(gdb)
\$1 = 1

(gdb)
\$2 = 0

(gdb)
After
reaches 4 and the loop terminates before processing 5.

Correct Condition
for (i = 1; i <= 5; i++)
Correct Output
Sum = 15
Conclusion

GDB helped us observe the values of i and sum during execution. The incorrect condition i < 5 was identified and replaced with i <= 5, producing the required result.



-g case4.c -o case4
GDB:
gdb ./case4
Display the source code:
list
breakpoint:
break 8
the program:
run
the variables:
print i

print sum

Continue execution:
continue
tracing the loop, it can be seen that the final value of i

CASE 5: DEBUGGING A FUNCTION USING NEXT AND STEP

Aim

To understand the difference between the GDB commands next and step while debugging a program containing a function call.

Program

```
#include <stdio.h>
```

```
int square(int x)
```

```
{
```

```
    int result;
```

```
    result = x * x;
```

```
    return result;
```

```
}
```

```
int main()
```

```
{
```

```
    int n = 5;
```

```
    int answer;
```

```
    answer = square(n);
```

```
    printf("Square = %d\n", answer);
```

```
    return 0;
```

```
}
```

Description

The function square() receives a number and returns its square. In this program, the value 5 is passed to the function, so the returned result is 25.

GDB provides two useful commands for moving through program execution:

Option A – Using next

The next command executes the current source statement. When it encounters a function call, GDB normally executes that function completely and stops at the following line in the current function.

Example:

```
(gdb) break main
```

```
(gdb) run
```

```
(gdb) next
```

```
(gdb) next
```

```
(gdb) print answer
```

```
$1 = 25
```

Here, GDB moves over the square() function call without entering each statement inside the function.

Option B – Using step

The step command is useful when we want to enter a function and inspect its internal statements.

Example:

```
(gdb) break main
```

```
(gdb) run
```

```
(gdb) step
```

When the execution reaches:

```
answer = square(n);
```

using step takes the debugger inside the square() function.

The value can then be inspected:

```
(gdb) print x
```

```
$1 = 5
```

```
(gdb) next
```

```
(gdb) print result
```

```
$2 = 25
```

After completing the function, execution returns to main().

Output

```
Square = 25
```

Difference Between next and step

Command

Working

next

Moves to the next line and normally skips over function internals.

step

Moves line-by-line and enters a function when a function call is encountered.

Conclusion

The next command is useful when the internal working of a function is not required, whereas step is useful when we need to inspect the statements inside the called function. Both commands provide controlled execution during debugging

CASE 6: DEBUGGING RUNTIME ERRORS USING BACKTRACE

AIM:

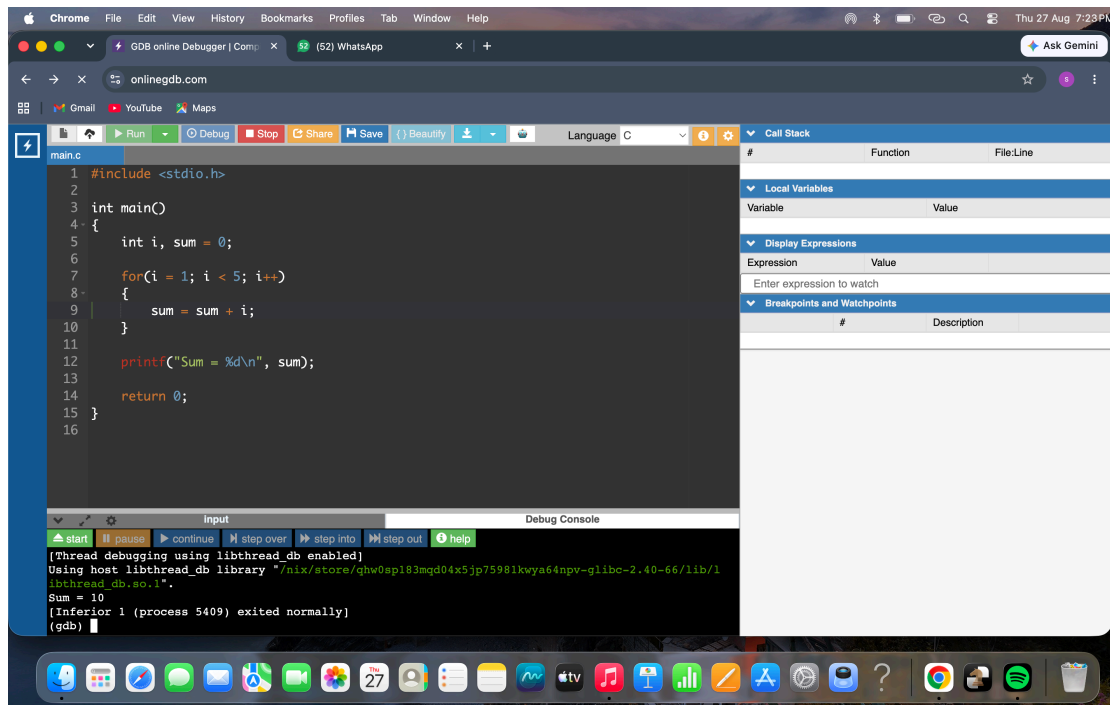
To identify the cause and location of a runtime error using the GDB backtrace command.

PROGRAM:

```
#include <stdio.h>
```

```
int divide(int a, int b)
```

```
{  
    return a / b;  
}
```



```
int calculate(int x)
{
    return divide(x, 0);
}
```

```
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

ERROR:

The function calculate() calls divide() with 0 as the divisor, causing a division-by-zero runtime error.

GDB EXECUTION:

gcc -g sample6.c -o sample6

gdb ./sample6

INSIDE GDB:

(gdb) run

Program received signal SIGFPE, Arithmetic exception.

(gdb) backtrace

#0 divide (a=10, b=0)

#1 calculate (x=10)

#2 main ()

FRAME INSPECTION:

(gdb) frame 0

(gdb) print b

\$1 = 0

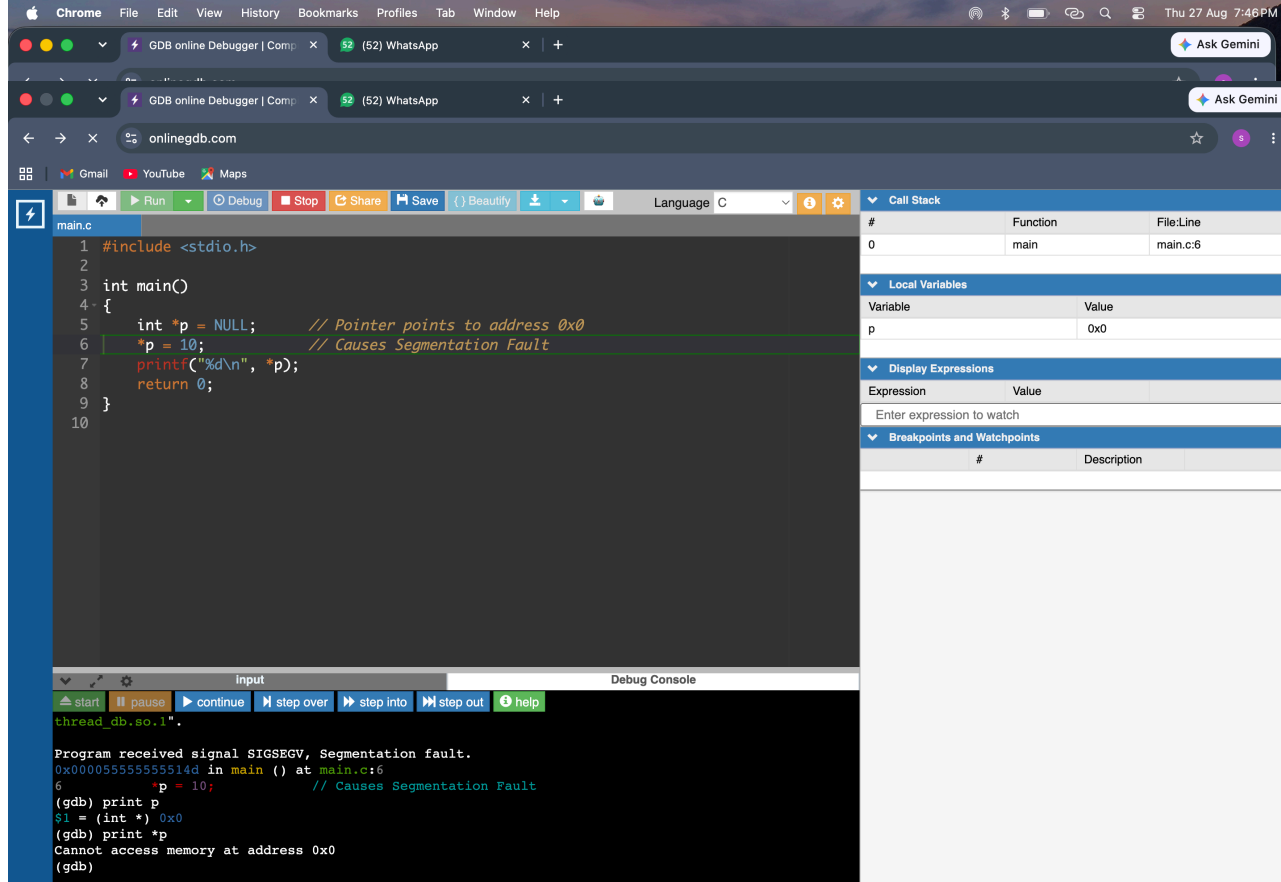
(gdb) frame 1

(gdb) print x

\$2 = 10

CONCLUSION:

The backtrace command identifies the exact location of the runtime error and shows the sequence of function calls. The error occurred because b = 0.



CASE 7: DEBUGGING ARRAY OUT OF BOUNDS USING GDB

AIM:

To identify and correct an array out-of-bounds error in a C program using GDB.

PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
    int a[5];
    int i;

    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]);
    }

```

```
    return 0;
}
```

ERROR:

The array a[5] has valid indexes from 0 to 4.

The condition `i <= 5` allows access to `a[5]`, which is outside the array.

GDB EXECUTION:

```
gcc -g sample7.c -o sample7
```

```
gdb ./sample7
```

INSIDE GDB:

```
(gdb) break 8
```

```
(gdb) run
```

```
(gdb) print &a[0]
```

```
$1 = (int *) 0x...
```

```
(gdb) print &a[4]
```

```
$2 = (int *) 0x...
```

```
(gdb) print &a[5]
```

```
$3 = (int *) 0x...
```

CORRECTED CODE:

Change:

```
for(i = 0; i <= 5; i++)
```

to:

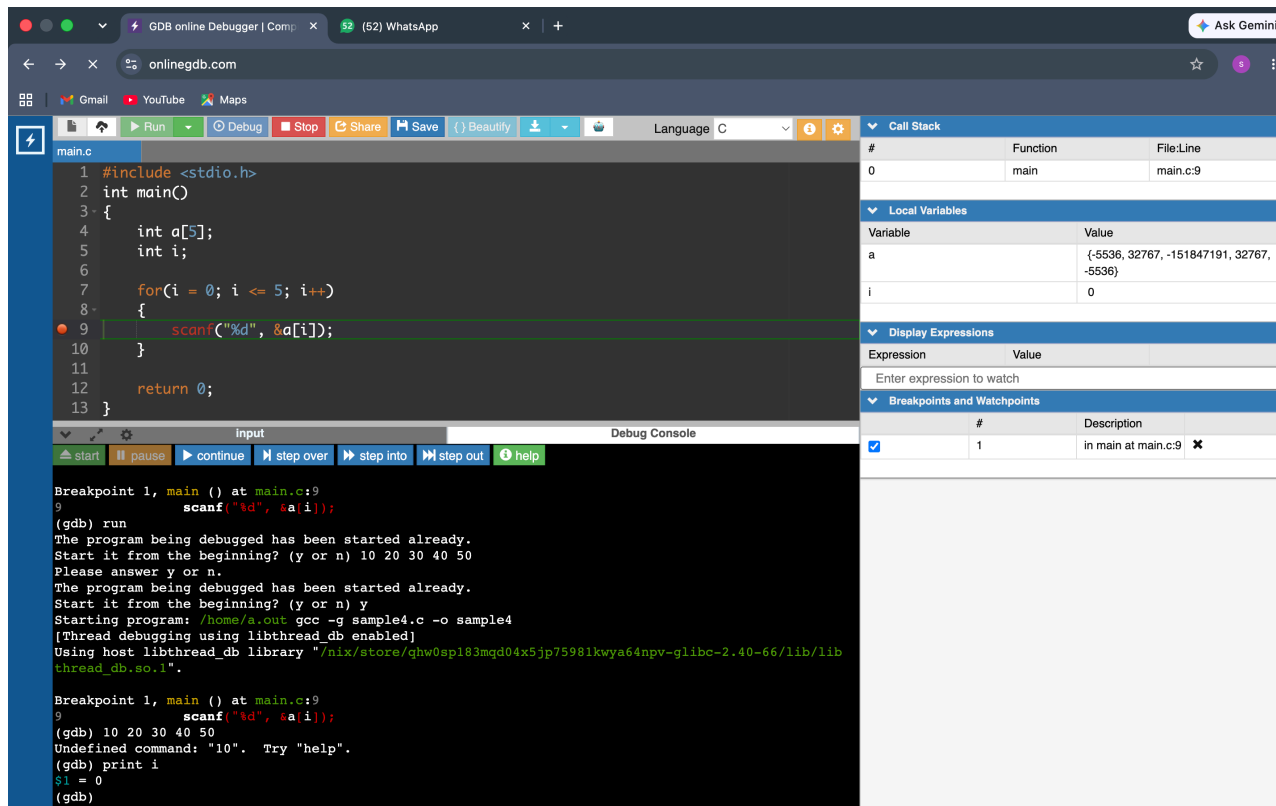
```
for(i = 0; i < 5; i++)
```

VALID INDEXES:

```
a[0], a[1], a[2], a[3], a[4]
```

CONCLUSION:

The error occurred because the loop allowed access to `a[5]`. Changing `i <= 5` to `i < 5` prevents the out-of-bounds access.



CASE 8: DEBUGGING SEGMENTATION FAULT USING GDB

AIM:

To identify and understand a segmentation fault caused by accessing an invalid memory address using GDB.

PROGRAM:

```
#include <stdio.h>
```

```

int main()
{
    int *p = NULL;

    *p = 10;

    printf("%d\n", *p);

    return 0;
}

```

ERROR:

The pointer p is NULL. Dereferencing it using *p = 10 causes a segmentation fault.

GDB EXECUTION:

```
gcc -g sample8.c -o sample8
```

```
gdb ./sample8
```

INSIDE GDB:

```
(gdb) run
```

Program received signal SIGSEGV, Segmentation fault.

```

(gdb) print p
$1 = (int *) 0x0

```

```

(gdb) print *p
Cannot access memory at address 0x0
CORRECTED CODE:
#include <stdio.h>

```

```

int main()
{

```



```
int x;  
int *p = &x;  
  
*p = 10;  
  
printf("%d\n", *p);  
  
return 0;  
}
```

OUTPUT:

10

CONCLUSION:

The segmentation fault was caused by dereferencing a NULL pointer. Assigning the pointer to a valid memory address fixes the error.